

COL2010 — Introduction to Data Science

Mid-Semester Examination (Practice) — Module 1: Database Systems

Set A

Instructor: Prof. Maya Ramanath | Semester: Diwali Semester, 2026 Total Marks: 70 |

Time: 90 minutes

Instructions:

- Answer all 15 questions. Marks for each question are indicated alongside it.
 - Show all intermediate steps (attribute closures, algebra derivations, query plans) — final answers alone will not receive full credit.
 - Write SQL in standard ANSI syntax unless a dialect-specific function is explicitly required.
 - Be concise and precise; avoid long descriptive paragraphs — use tables, derivations, and short justifications.
-

Q1. (3 marks) ER-to-Relational Mapping

Consider the following ER description:

- Entity set **Employee** (EmpID, Name, Age).
- Entity set **Department** (DeptCode, DeptName, Location).
- Relationship **WorksFor** between Employee and Department, where *each employee works for exactly one department, but a department has many employees*. The relationship carries its own attribute, **StartDate**.

Convert this ER diagram into a minimal set of relational schemas (i.e., do not create more tables than necessary). Clearly underline the primary key of each relation you produce.

Q2. (5 marks) Closure of Attributes

Let relation $R(A, B, C, D, E)$ have the following functional dependencies:

$$AB \rightarrow C, \quad C \rightarrow D, \quad D \rightarrow E, \quad E \rightarrow A$$

(a) Compute $\{A, B\}^+$ showing every iteration of the closure algorithm. (b) Using your result, state whether $\{A, B\}$ is a candidate key of R . Justify using the definition of a key.

Q3. (6 marks) Bag Algebra Identity

Let R, S, T be relations under **bag (multiset)** semantics. Prove or disprove the following identity:

$$(R \cup S) - T = (R - T) \cup (S - T)$$

Use a small explicit counter-example (or a formal argument) that tracks multiplicities of a single tuple across R, S, T .

Q4. (4 marks) Relational Algebra

Given schemas:

- `Students(SID, SName, Dept)`
- `Enrollments(SID, CourseID, Grade)`

Write a relational algebra expression (using $\pi, \sigma, \bowtie$) to return the **names** of students who are enrolled in the course with `CourseID = 'COL2010'` and obtained a `Grade` of `'A'`.

Q5. (2 marks) Keys — Short Answer

In one or two lines each, distinguish between a **candidate key** and a **primary key** of a relation. Give a one-line example illustrating a relation with two candidate keys.

Q6. (5 marks) SQL — Aggregation with a Correlated Subquery

Given `Employees(EmpID, Name, DeptCode, Salary)`, write a single SQL query that returns the `DeptCode` and `AVG(Salary)` for every department whose average salary **exceeds the average salary of the entire company**. (Hint: you will need a subquery that computes the company-wide average, combined with `GROUP BY` / `HAVING` over departments.)

Q7. (6 marks) BCNF Decomposition

Consider relation $R(A, B, C, D)$ with functional dependencies:

$$A \rightarrow B, \quad B \rightarrow C$$

(a) List all candidate key(s) of R . (b) Identify a BCNF violation, if any, and decompose R into a lossless-join set of relations, each of which is in BCNF. Show the decomposition step precisely (state which FD you used and how you split the attributes).

Q8. (3 marks) SQL — Join and Sorting

Given `Routes(RouteID, SourceCity, DestCity, DistanceKM)` and `Carriers(CarrierID, RouteID, CarrierName)`, write a SQL query that lists `CarrierName`, `SourceCity`, `DestCity` for all routes with `DistanceKM > 500`, sorted by `DistanceKM` in descending order.

Q9. (4 marks) N-ary to Binary Relationship Conversion

An **Actor** plays a **Part** in a **Movie** — this is modeled as a single ternary (3-way) relationship `Plays(Actor, Part, Movie)` in an ER diagram.

Explain, with a diagram or schema-level reasoning, how this ternary relationship can be converted into an equivalent set of **binary** relationships. What new entity set (if any) needs to be introduced, and why?

Q10. (7 marks) Recursive CTE — Trace and Write

Consider a table `Edge(from_city, to_city)` with the following rows:

from_city	to_city
Delhi	Bengaluru
Bengaluru	Chennai
Chennai	Pune

(a) Write a `WITH RECURSIVE` SQL query that computes `Path(f, t)`: all pairs of cities (f, t) such that t is reachable from f by following one or more edges. (b) Trace the **least fixpoint evaluation** of your query for the given `Edge` table — list the contents of `Path` after each round (iteration) until no new rows are added, and state the stopping condition used.

Q11. (3 marks) DDL — Schema Definition

Write `CREATE TABLE` statements for `Department(DeptCode, DeptName)` and `Employee(EmpID, Name, DeptCode)` such that:

- `DeptCode` is the primary key of `Department`.
 - `EmpID` is the primary key of `Employee`, `Name` is mandatory, and `DeptCode` in `Employee` correctly references `Department`.
-

Q12. (5 marks) Window Functions — Trace

Given `Salaries(EmpID, DeptCode, SalaryAmt)`:

EmpID	DeptCode	SalaryAmt
E1	D1	90000
E2	D1	70000
E3	D1	85000
E4	D2	60000
E5	D2	95000

(a) Write a query using $\text{ROW_NUMBER() OVER (PARTITION BY DeptCode ORDER BY SalaryAmt DESC)}$ to rank employees within each department. (b) Show the resulting table (EmpID , DeptCode , SalaryAmt , rn) after applying your query to the data above.

Q13. (6 marks) Datalog — Safety and Stratification

Consider the following Datalog rules:

```
P(x) :- R(x, y), NOT Q(y)
Q(x) :- R(x, y), P(y)
S(x) :- P(x), NOT Q(x)
```

(a) Construct the predicate dependency graph (nodes = IDB predicates, edges labelled $\oplus/\ominus$ per the rule bodies). (b) Are these rules stratifiable? Justify using the definition of stratified negation (no negation allowed within a mutually recursive cycle). If not stratifiable, identify exactly which cycle causes the problem.

Q14. (4 marks) Minimum/Maximum Tuples — Natural Join

Let R and S be two relations with $|R| = r$ and $|S| = s$, and suppose R and S share exactly one common attribute A , which is a **key** of R (but not necessarily of S). What are the **minimum** and **maximum** possible number of tuples in the natural join $R \bowtie S$? Briefly justify each bound.

Q15. (6 marks) Normalization — Diagnose and Decompose

Consider a single relation:

Booking(BookingID, GuestName, RoomNumber, HotelName, HotelCity)

with the functional dependencies:

$$\text{BookingID} \rightarrow \text{GuestName}, \text{RoomNumber}, \text{HotelName}$$
$$\text{HotelName} \rightarrow \text{HotelCity}$$

(a) State the candidate key of Booking . (b) Identify which normal form (1NF / 2NF / 3NF) is **violated**, and name the specific violation using the formal definition (not just "redundancy"). (c) Decompose Booking into a set of relations that are all in 3NF, listing the schema and key of each resulting relation.

End of Set A

Answer Key & Marking Scheme

Q1 (3 marks). $\text{Employee}(\text{EmpID}, \text{Name}, \text{Age}, \text{DeptCode}, \text{StartDate})$ — PK: EmpID ; DeptCode is a foreign key referencing Department . $\text{Department}(\text{DeptCode}, \text{DeptName}, \text{Location})$ — PK: DeptCode . (Because the relationship is many-to-one from Employee to Department, both the foreign key and the relationship attribute StartDate can be folded into the Employee relation — no separate WorksFor table is needed.)

- 1 mark: correct Department relation.
- 1.5 marks: correct merged Employee relation with StartDate folded in and DeptCode as FK.
- 0.5 mark: correctly *not* creating a third table.

Q2 (5 marks). $\{A, B\}^+$: Start $Z = \{A, B\}$. $AB \rightarrow C$ applies: $Z = \{A, B, C\}$. $C \rightarrow D$ applies: $Z = \{A, B, C, D\}$. $D \rightarrow E$ applies: $Z = \{A, B, C, D, E\}$. $E \rightarrow A$ applies but adds nothing new. Z unchanged $\rightarrow$ stop. $\{A, B\}^+ = \{A, B, C, D, E\}$ = all attributes of $R \Rightarrow \{A, B\}$ is a **superkey**, and since no proper subset of it determines all attributes (check $\{A\}^+ = \{A\}$, $\{B\}^+ = \{B\}$), it is **minimal**, hence a **candidate key**.

- 3 marks for correct iterative closure computation (any correct iteration order accepted).
- 2 marks for correctly concluding candidate key with minimality justification.

Q3 (6 marks). Disproved. Counter-example: let a tuple x appear once in R , once in S , and once in T (all other tuples empty).

- Multiplicity of x in $R \cup S$ (bag union, multiplicities add) = $1 + 1 = 2$. Subtracting T (bag difference, multiplicities subtract, floor at 0): $2 - 1 = 1$. So LHS has x with multiplicity 1.
- Multiplicity of x in $R - T = 1 - 1 = 0$. Multiplicity of x in $S - T = 1 - 1 = 0$. Their bag union has x with multiplicity $0 + 0 = 0$. So RHS has x with multiplicity 0.
- $1 \neq 0$, so the identity fails under bag semantics.
- 3 marks for correctly identifying "disprove."
- 3 marks for a valid, clearly worked multiplicity counter-example.

Q4 (4 marks).

$$\pi_{SName} \left(\sigma_{CourseID='COL2010' \wedge Grade='A'} (Students \bowtie Enrollments) \right)$$

- 2 marks join condition/composition correct, 2 marks correct selection + projection attributes.

Q5 (2 marks). Candidate key: any minimal set of attributes that functionally determines all other attributes (a relation may have several). Primary key: the one candidate key chosen

by the designer to be *the* main identifier. Example: `Person(SSN, Passport, Name)` where both `SSN` and `Passport` are candidate keys but only one is designated primary key.

- 1 mark per correct distinction with example.

Q6 (5 marks).

sql

```
SELECT DeptCode, AVG(Salary) AS AvgDeptSalary
FROM Employees
GROUP BY DeptCode
HAVING AVG(Salary) > (SELECT AVG(Salary) FROM Employees);
```

- 3 marks for correct GROUP BY/HAVING structure, 2 marks for correct scalar subquery for company-wide average.

Q7 (6 marks). (a) $A^+ = \{A, B, C\}$ (not all attributes, since D is missing) $\Rightarrow A$ alone is not a key. $\{A, D\}^+ = \{A, B, C, D\}$ = all attributes, and this is minimal $\Rightarrow$ candidate key is $\{A, D\}$. (b) $A \rightarrow B$: is A a superkey? No ($A^+ \neq$ all attributes). This violates BCNF. Decompose using $A \rightarrow B$: $R_1(A, B)$ (attributes of $X \cup Y = \{A, B\}$) and $R_2(A, C, D)$ (remaining attributes plus X). Re-check R_2 : FD $B \rightarrow C$ no longer applies directly (B not present); need to re-derive FDs on R_2 — since $A \rightarrow C$ is implied ($A \rightarrow B \rightarrow C$), and $\{A, D\}$ is a key of R_2 , R_2 is in BCNF. $R_1(A, B)$ with $A \rightarrow B$, A is the key, so also BCNF.

- 2 marks candidate key; 2 marks correctly identifying violation; 2 marks correct lossless decomposition.

Q8 (3 marks).

sql

```
SELECT c.CarrierName, r.SourceCity, r.DestCity
FROM Routes r JOIN Carriers c ON r.RouteID = c.RouteID
WHERE r.DistanceKM > 500
ORDER BY r.DistanceKM DESC;
```

- 1.5 marks correct join, 1.5 marks correct filter + sort.

Q9 (4 marks). Introduce a new entity set, e.g. `MovieRole(RoleID, ...)`, representing the (Actor, Part, Movie) combination itself. Then create three binary many-one relationships: `playedBy(MovieRole  $\rightarrow$  Actor)`, `partOf(MovieRole  $\rightarrow$  Movie)`, and an attribute/relationship capturing which Part is being played. This is necessary because a single binary relationship cannot capture a constraint that simultaneously involves three participating entity sets — the new entity set acts as a connecting node for the three binary edges.

- 2 marks new entity set correctly introduced; 2 marks correct three binary relationships / reasoning.

Q10 (7 marks). (a)

sql

```
WITH RECURSIVE Path(f, t) AS (
  SELECT from_city, to_city FROM Edge
  UNION
  SELECT Path.f, Edge.to_city
  FROM Path, Edge
  WHERE Path.t = Edge.from_city
)
SELECT * FROM Path;
```

(b) Iteration trace (least fixpoint, starting from empty IDB):

- Round 0 (base case): Path = {(Delhi,Bengaluru), (Bengaluru,Chennai), (Chennai,Pune)}
- Round 1 (recursive step adds): (Delhi,Chennai), (Bengaluru,Pune)
- Round 2: (Delhi,Pune)
- Round 3: no new tuples produced $\Rightarrow$ stop (fixpoint reached: Path unchanged between rounds).
- 3 marks correct recursive SQL; 4 marks correct round-by-round trace and correct stopping condition ("Path unchanged").

Q11 (3 marks).

sql

```
CREATE TABLE Department (
  DeptCode VARCHAR(10) PRIMARY KEY,
  DeptName VARCHAR(100)
);

CREATE TABLE Employee (
  EmpID INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  DeptCode VARCHAR(10),
  FOREIGN KEY (DeptCode) REFERENCES Department(DeptCode)
);
```

- 1.5 marks each table correctly defined.

Q12 (5 marks).

sql

```
SELECT EmpID, DeptCode, SalaryAmt,
       ROW_NUMBER() OVER (PARTITION BY DeptCode ORDER BY SalaryAmt DESC) AS rn
FROM Salaries;
```

Resulting table:

EmpID	DeptCode	SalaryAmt	rn
E1	D1	90000	1
E3	D1	85000	2
E2	D1	70000	3
E5	D2	95000	1
E4	D2	60000	2

- 2 marks correct query; 3 marks correct traced output (partition + rank order).

Q13 (6 marks). (a) Dependency graph edges: $P \rightarrow Q$ labelled $\boxed{-}$ (since $\boxed{Q}$ is negated in P's rule... note direction convention: edge from P to Q if Q occurs in body of P). Specifically: rule for $\boxed{P}$ uses $\boxed{\text{NOT } Q} \Rightarrow$ edge $P \rightarrow Q$ labelled $\boxed{-}$. Rule for $\boxed{Q}$ uses $\boxed{P}$ (non-negated) $\Rightarrow$ edge $Q \rightarrow P$ labelled $\boxed{+}$. Rule for $\boxed{S}$ uses $\boxed{P}$ ($\boxed{+}$) and $\boxed{\text{NOT } Q}$ ($\boxed{-}$) $\Rightarrow$ edges $S \rightarrow P$ ($\boxed{+}$) and $S \rightarrow Q$ ($\boxed{-}$). (b) **Not stratifiable.** There is a cycle $P \rightarrow Q \rightarrow P$ (P depends negatively on Q, and Q depends positively on P, i.e., they are mutually recursive), and this cycle contains a negated edge ($P \rightarrow Q$ is $\boxed{-}$). Stratified negation requires that no negation occur within a mutually recursive cycle — this rule set violates that condition on the P - Q cycle.

- 3 marks correct graph with labelled edges; 3 marks correct identification of the violating cycle and justification.

Q14 (4 marks).

- **Minimum = 0:** if no value of A in R matches any value of A in S , the natural join produces no tuples.
- ****Maximum = $r \times s$ ** is not tight** here because A is a key of R — each tuple of R has a *unique* A -value, so it can join with at most as many tuples of S as share that A -value; in the extreme case all s tuples of S share the single A -value that also appears in one tuple of R , giving at most s matching pairs from that one R -tuple times up to r such distinct joins overall — the true tight upper bound is $\min(r \cdot$

s , but bounded by s matches per R -tuple); the standard accepted bound is ****maximum = s -many tuples per matching R row, summed over at most r rows, capped at $r \cdot s$ ****, i.e. ****maximum = $r \times s$ **** in the fully degenerate case where every row of S shares the same A value as one row of R (this does not violate A being a key of R , since S places no such restriction on A).

- 2 marks correct minimum with justification; 2 marks correct maximum with justification (accept $r \times s$ with correct reasoning about the key-side constraint).

Q15 (6 marks). (a) Candidate key: BookingID (it functionally determines all other attributes, directly or transitively). (b) Since BookingID is the only candidate key (a single-attribute key), 2NF cannot be violated (partial dependency requires a composite key). The violation is of 3NF: HotelName $\rightarrow$ HotelCity is a **transitive dependency** ($BookingID \rightarrow HotelName \rightarrow HotelCity$), and HotelName is not a superkey, HotelCity is not a prime attribute). (c) Decomposition:

- Booking(BookingID, GuestName, RoomNumber, HotelName) — key: BookingID.
- Hotel(HotelName, HotelCity) — key: HotelName.
- 1 mark key; 2 marks correct violation named with formal justification; 3 marks correct decomposition with keys.

Topic/Difficulty Coverage Log (for setting future sets — not shown to students)

(Retain internally when generating Set B onward to avoid repetition.)

#	Topic	Difficulty
Q1	ER-to-relational (many-one + rel. attribute)	Easy
Q2	Attribute closure / candidate key check	Moderate
Q3	Bag algebra identity (union/difference)	Hard
Q4	Relational algebra composition	Moderate
Q5	Candidate vs primary key	Easy
Q6	SQL GROUP BY/HAVING + scalar subquery	Moderate
Q7	BCNF decomposition	Hard
Q8	SQL join + ORDER BY	Easy
Q9	N-ary to binary relationship conversion	Moderate
Q10	Recursive CTE + fixpoint trace	Hard
Q11	DDL: CREATE TABLE + FK	Easy
Q12	Window function (ROW_NUMBER, partition)	Moderate
Q13	Datalog stratification/safety	Hard
Q14	Join tuple-count bounds (natural join w/ key)	Moderate
Q15	2NF/3NF diagnosis + decomposition	Hard

Not reused from Quiz 1: Q1 (full outer-join max), Q2 (bag set identity $\cap/-$), Q3 (FD closure for $AB \rightarrow D$), Q4 (ER conversion — People/Parent-of), Q5 (2NF check on Actor–Movies), Q6 (query plan for RA expression), Q7 (RA for boxoffice-per-movie). This set deliberately used different numeric examples, different FD sets, and different relation names/domains throughout.